

CHAPTER 17

8051 ADDRESSING MODES

OBJECTIVES

In this chapter, you will learn

- List the five addressing modes of the 8051 microcontroller.
- Contrast and compare the addressing modes.
- Code 8051 Assembly language instructions using each addressing mode.
- Access RAM using various addressing modes.
- List the SFR (special function registers) addresses.
- Discuss how to access the SFR.
- Manipulate the stack using direct addressing mode.
- Code 8051 instructions to manipulate a look-up table.
- Access RAM, I/O, and ports using bit addresses.
- Discuss how to access the extra 128 bytes of RAM space in the 8052.

The CPU can access data in various ways. The data could be in a register, or in memory, or be provided as an immediate value. These various ways of accessing data are called *addressing modes*. In this chapter we discuss 8051/52 addressing modes in the context of some examples.

The various addressing modes of a microprocessor are determined when it is designed, and therefore cannot be changed by the programmer. The 8051 provides a total of five distinct addressing modes. They are as follows:

1. immediate
2. register
3. direct
4. register indirect
5. indexed

In Section 17-1 we look at immediate and register addressing modes. In Section 17-2 we cover accessing memory using the direct, register indirect, and indexed addressing modes. Section 17-3 discusses the bit-addressability of RAM, registers, and I/O ports. In Section 17-4 we show how to access the extra 128 bytes of RAM in the 8052.

SECTION 17.1: IMMEDIATE AND REGISTER ADDRESSING MODES

In this section, first we examine immediate addressing mode and then register addressing mode.

17.1.1: Immediate addressing mode

In this addressing mode, the source operand is a constant. In immediate addressing mode, as the name implies, when the instruction is assembled, the operand comes immediately after the opcode. Notice that the immediate data must be preceded by the pound sign, "#". This addressing mode can be used to load information into any of the registers, including the DPTR register. Examples follow.

```
MOV A, #25H      ;load 25H into A
MOV R4, #62      ;load the decimal value 62 into R4
MOV B, #40H      ;load 40H into B
MOV DPTR, #4521H ;DPTR=4512H
```

Although the DPTR register is 16-bit, it can also be accessed as two 8-bit registers, DPH and DPL, where DPH is the high byte and DPL is the low byte. Look at the following code.

```
MOV DPTR, #2550H
    is the same as:
MOV DPL, #50H
MOV DPH, #25H
```

Also notice that the following would produce an error since the value is larger than 16 bits.

```
MOV DPTR, #68975; illegal!! value > 65535 (FFFFH)
```

We can use the EQU directive to access immediate data as shown below.

```
COUNT    EQU 30
...
MOV R4, #COUNT      ;R4=1E(30=1EH)
MOV DPTR, #MYDATA     ;DPTR=200H

ORG 200H
MYDATA: DB "America"
```

Notice that we can also use immediate addressing mode to send data to 8051 ports. For example, "MOV P1, #55H" is a valid instruction.

17.1.2: Register addressing mode

Register addressing mode involves the use of registers to hold the data to be manipulated. Examples of register addressing mode follow.

```
MOV A, R0 ; copy the contents of R0 into A
MOV R2, A ; copy the contents of A into R2
ADD A, R5 ; add the contents of R5 to contents of A
ADD A, R7 ; add the contents of R7 to contents of A
MOV R6, A ; save accumulator in R6
```

It should be noted that the source and destination registers must match in size. In other words, coding "MOV DPTR, A" will give an error, since the source is an 8-bit register and the destination is a 16-bit register. See the following.

```
MOV DPTR, #25F5H
MOV R7, DPL
MOV R6, DPH
```

Notice that we can move data between the accumulator and Rn (for n = 0 to 7) but movement of data between Rn registers is not allowed. For example, the instruction "MOV R4, R7" is invalid.

In the first two addressing modes, the operands are either inside one of the registers or tagged along with the instruction itself. In most programs, the data to be processed is often in some memory location of RAM or in the code space of ROM. There are many ways to access this data. The next section describes these different methods.

17.1.3: Review problems

- Can the programmer of a microcontroller make up new addressing modes?
- Show the instruction to load 1000 0000 (binary) into R3.
- Why is the following invalid? "MOV R2, DPTR"
- True or false. DPTR is a 16-bit register that is also accessible in low-byte and high-byte formats.
- Is the PC (program counter) also available in low-byte and high-byte formats?

SECTION 17.2: ACCESSING MEMORY USING VARIOUS ADDRESSING MODES

We can use direct or register indirect addressing modes to access data stored either in RAM or registers of the 8051. This topic will be discussed thoroughly in this section. We will also show how to access on-chip ROM containing data using indexed addressing mode.

17.2.1: Direct addressing mode

As mentioned in Chapter 14, there are 128 bytes of RAM in the 8051. The RAM has been assigned addresses 00 to 7FH. The following is a summary of the allocation of these 128 bytes.

- RAM locations 00 - 1FH are assigned to the register banks and stack.
- RAM locations 20 - 2FH are set aside as bit-addressable space to save single-bit data. This is discussed in Section 17-3.
- RAM locations 30 - 7FH are available as a place to save byte-sized data.

Although the entire 128 bytes of RAM can be accessed using direct addressing mode, it is most often used to access RAM locations 30 - 7FH. This is due to the fact that register bank locations are accessed by the register names of R0 - R7, but there is no such name for other RAM locations. In the direct addressing mode, the data is in a RAM memory location whose address is known, and this address is given as a part of the instruction. Contrast this with immediate addressing mode, in which the operand itself is provided with the instruction. The "#" sign distinguishes between the two modes. See the examples below, and note the absence of the "#" sign.

```
MOV R0, 40H ; save content of RAM location 40H in R0
MOV 56H, A ; save content of A in RAM location 56H
MOV R4, 7FH ; move contents of RAM location 7FH to R4
```


As discussed earlier, RAM locations 0 to 7 are allocated to bank 0 registers R0 - R7. These registers can be accessed in two ways, as shown below.

```
MOV A, 4      ;is same as
MOV A, R4     ;which means copy R4 into A

MOV A, 7      ;is same as
MOV A, R7     ;which means copy R7 into A

MOV A, 2      ;is the same as
MOV A, R2     ;which means copy R2 into A

MOV A, 0      ;is the same as
MOV A, R0     ;which means copy R0 into A
```

The above examples should reinforce the importance of the "#" sign in 8051 instructions. See the following code.

```
MOV R2, #5    ;R2 with value 5
MOV A, 2      ;copy R2 to A (A=R2=05)
MOV B, 2      ;copy R2 to B (B=R2=05)
MOV 7, 2      ;copy R2 to R7
               ;since "MOV R7, R2" is invalid
```

Although it is easier to use the names R0 - R7 than their memory addresses, RAM locations 30H to 7FH cannot be accessed in any way other than by their addresses since they have no names.

17.2.2: SFR registers and their addresses

Among the registers we have discussed so far, we have seen that R0 - R7 are part of the 128 bytes of RAM memory. What about registers A, B, PSW, and DPTR? Do they also have addresses? The answer is yes. In the 8051, registers A, B, PSW, and DPTR are part of the group of registers commonly referred to as SFR (special function registers). There are many special function registers and they are widely used, as we will discuss in future chapters. The SFR can be accessed by their names (which is much easier) or by their addresses. For example, register A has address E0H, and register B has been designated the address F0H, as shown in Table 17-1. Notice how the following pairs of instructions mean the same thing.

```
MOV 0E0H, #55H ;is the same as
MOV A, #55H    ;which means load 55H into A (A=55H)

MOV 0F0H, #25H ;is the same as
MOV B, #25H    ;which means load 25H into B (B=25H)

MOV 0E0H, R2   ;is the same as
MOV A, R2      ;which means copy R2 into A

MOV 0F0H, R0   ;is the same as
MOV B, R0      ;which means copy R0 into B

MOV P1, A      ;is the same as
MOV 90H, A     ;which means copy reg A to P1
```

Table 17-1 lists the 8051 special function registers (SFR) and their addresses. The following two points should be noted about the SFR addresses.

1. The special function registers have addresses between 80H and FFH. These addresses are above 80H, since the addresses 00 to 7FH are addresses of RAM memory inside the 8051.
2. Not all the address space of 80 to FF is used by the SFR. The unused locations 80H to FFH are reserved and must not be used by the 8051 programmer.

Regarding direct addressing mode, notice the following two points: (a) the address value is limited to one byte 00 - FFH, which means this addressing mode is limited to accessing RAM locations and registers located inside the 8051. (b) if you examine the 1st file for an Assembly language program, you will see that the SFR registers' names are replaced with their addresses as listed in Table 17-1.

Table 17-1: 8051 Special Function Register (SFR) Addresses

Symbol	Name	Address
ACC*	Accumulator	0E0H
B*	B register	0F0H
PSW*	Program status word	0D0H
SP*	Stack pointer	81H
DPTR	Data pointer 2 bytes	
DPL	Low byte	82H
DPH	High byte	83H
P0*	Port 0	80H
P1*	Port 1	90H
P2*	Port 2	0A0H
P3*	Port 3	0B0H
IP*	Interrupt priority control	0B8H
IE*	Interrupt enable control	0A8H
TMOD	Timer/counter mode control	89H
TCON*	Timer/counter control	88H
T2CON*	Timer/counter 2 control	0C8H
T2MOD	Timer/counter mode control	0C9H
TH0	Timer/counter 0 high byte	8CH
TL0	Timer/counter 0 low byte	8AH
TH1	Timer/counter 1 high byte	8DH
TL1	Timer/counter 1 low byte	8BH
TH2	Timer/counter 2 high byte	0CDH
TL2	Timer/counter 2 low byte	0CCH
RCAP2H	T/C 2 capture register high byte	0CBH
RCAP2L	T/C 2 capture register low byte	0CAH
SCON*	Serial control	98H
SBUF	Serial data buffer	99H
PCON	Power control	87H

* Bit-addressable

Example 17-1

Write code to send 55H to ports P1 and P2, using (a) their names, (b) their addresses.

Solution:

(a) `MOV A, #55H` ; A=55H
`MOV P1, A` ; P1=55H
`MOV P2, A` ; P2=55H

(b) From Table 5-1, P1 address = 90H; P2 address = A0H
`MOV A, #55H` ; A=55H
`MOV 90H, A` ; P1=55H
`MOV 0A0H, A` ; P2=55H

Example 17-2

Transfer the contents of the registers A, R0, and R1, respectively, of bank 0 to the registers B, R0, and R1, of bank 1 using stack operations.

Solution:

On reset, the stack pointer is initialized to 07.

Let us now initialize it to 3FH so that the first stack location is 40H and the stack location does not create a conflict with bank 1 registers.

```
MOV SP, #3FH          ; SP=3FH
PUSH 00               ; push R0 of bank 0 to stack top
PUSH 01               ; push R1 of bank 0 to stack
PUSH 0E0H             ; push A register to stack
SETB PSW.1           ; switch to bank 1 by making RS0 bit ; of PSW (PSW.1)=1
POP 0F0H              ; pop the content of stack top to B register
POP 09                ; pop the content of stack top to R1 of bank 1
POP 08                ; pop the content of stack top to R0 of bank 1
```

Now we have the content of A in B, and the contents of bank 0 registers R0 and R1 are transferred to bank 1 registers R0 and R1.

17.2.3: Stack and direct addressing mode

Another major use of direct addressing mode is the stack. In the 8051 family, only direct addressing mode is allowed for pushing onto the stack. Therefore, an instruction such as "PUSH A" is invalid. Pushing the accumulator onto the stack must be coded as "PUSH 0E0H" where 0E0H is the address of register A. Similarly, pushing R3 of bank 0 is coded as "PUSH 03". Direct addressing mode must be used for the POP instruction as well. For example, "POP 04" will pop the top of the stack into R4 of bank 0.

17.2.4: Register indirect addressing mode

In the register indirect addressing mode, a register is used as a pointer to the data. If the data is inside the CPU, only registers R0 and R1 are used for this purpose. In other words, R2 - R7 cannot be used to hold the address of an operand located in RAM when using this addressing mode. When R0 and R1 are used as pointers, that is, when they hold the addresses of RAM locations, they must be preceded by the "@" sign, as shown below.

```
MOV A, @R0            ; move contents of RAM location whose
                      ; address is held by R0 into A
MOV @R1, B             ; move contents of B into RAM location
                      ; whose address is held by R1
```

Notice that R0 (as well as R1) is preceded by the "@" sign. In the absence of the "@" sign, MOV will be interpreted as an instruction moving the contents of register R0 to A, instead of the contents of the memory location pointed to by R0.

17.2.5: Advantage of register indirect addressing mode

One of the advantages of register indirect addressing mode is that it makes accessing data dynamic rather than static as in the case of direct addressing mode. Example 17-3 shows two cases of copying 55H into RAM locations

40H to 45H. Notice in solution (b) that there are two instructions that are repeated numerous times. We can create a loop with those two instructions as shown in solution (c). Solution (c) is the most efficient and is possible only because of register indirect addressing mode. Looping is not possible in direct addressing mode. This is the main difference between the direct and register indirect addressing modes.

Example 17-3

Write a program to copy the value 55H into RAM memory locations 40H to 45H using

- (a) direct addressing mode,
- (b) register indirect addressing mode without a loop, and
- (c) with a loop.

Solution:

```
(a)      MOV    A, #55H           ;load A with value 55H
          MOV    40H, A          ;copy A to RAM location 40H
          MOV    41H, A          ;copy A to RAM location 41H
          MOV    42H, A          ;copy A to RAM location 42H
          MOV    43H, A          ;copy A to RAM location 43H
          MOV    44H, A          ;copy A to RAM location 44H

(b)      MOV    A, #55H           ;load A with value 55H
          MOV    R0, #40H        ;load the pointer. R0=40H
          MOV    @R0, A          ;copy A to RAM location R0 points to
          INC    R0              ;increment pointer. Now R0=41H
          MOV    @R0, A          ;copy A to RAM location R0 points to
          INC    R0              ;increment pointer. Now R0=42H
          MOV    @R0, A          ;copy A to RAM location R0 points to
          INC    R0              ;increment pointer. Now R0=43H
          MOV    @R0, A          ;copy A to RAM location R0 points to
          INC    R0              ;increment pointer. Now R0=44H
          MOV    @R0, A          ;increment pointer. Now R0=44H

(c)      MOV    A, #55           ;A=55H
          MOV    R0, #40H        ;load pointer. R0=40H, RAM address
          MOV    R2, #05         ;load counter, R2=5
AGAIN:   MOV    @R0, A          ;copy 55H to RAM location R0 points to
          INC    R0              ;increment R0 pointer
          DJNZ   R2, AGAIN       ;loop until counter = zero
```

Example 17-4

Write a program to clear 16 RAM locations starting at RAM address 60H.

Solution:

```
          CLR    A               ;A=0
          MOV    R1, #60H        ;load pointer, R1=60H
          MOV    R7, #16         ;load counter, R7=16 (10 in hex)
AGAIN:   MOV    @R1, A          ;clear RAM location R1 points to
          INC    R1              ;increment R1 pointer
          DJNZ   R7, AGAIN       ;loop until counter = zero
```


Example 17-5

Take 10 bytes of data from data RAM locations 45H to 54H, add 02 to each of them, and save the result in data RAM locations 79H down to 70H.

Solution:

```
BACK:  MOV R2, #10      ;R2 is used as the counter
        MOV R0, #45H   ;R0 points to first source location
        MOV R1, #79H   ;R1 points to first destination location
        MOV A, @R0     ;move source byte to A
        ADD A, #02     ;add 2 to it
        MOV @R1, A     ;move it to destination
        INC R0         ;increment the source pointer
        DEC R1         ;decrement the destination pointer
        ;this is because the address of destination is in
        ;decreasing order
        DJNZ R2, BACK  ;repeat until counter value is zero
```

An example of how to use both R0 and R1 in the register indirect addressing mode in a block transfer is given in Example 17-5.

17.2.6: Limitation of register indirect addressing mode in the 8051

As stated earlier, R0 and R1 are the only registers that can be used for pointers in register indirect addressing mode. Since R0 and R1 are 8 bits wide, their use is limited to accessing any information in the internal RAM (scratch pad memory of 30H - 7FH, or SFR). However, there are times when we need to access data stored in external RAM or in the code space of on-chip ROM. Whether accessing externally connected RAM or on-chip ROM, we need a 16-bit pointer. In such cases, the DPTR register is used, as shown next.

17.2.7: Indexed addressing mode and on-chip ROM access

Indexed addressing mode is widely used in accessing data elements of look-up table entries located in the program ROM space of the 8051. The instruction used for this purpose is "MOVC A, @A+DPTR". The 16-bit register DPTR and register A are used to form the address of the data element stored in on-chip ROM. Because the data elements are stored in the program (code) space ROM of the 8051, the instruction MOVC is used instead of MOV. The "C" means code. In this instruction the contents of A are added to the 16-bit register DPTR to form the 16-bit address of the needed data. See Example 17-6.

Example 17-6

The word "SAM" is to be burned in the flash ROM location starting from 0400H of an AT89C51. Write a program to do this and to read this data into internal RAM locations starting from 60H.

Solution:

```
ORG 0000H
CLR A      ;program starts at location 0000H
MOV DPTR, #0400H ;A=0.
MOVC A, @A+DPTR ;DPTR=400H (points to first source location)
MOV 60H, A    ;get 'S' from location 400H
              ;move it to RAM location 60H
```

```

INC DPTR
CLR A
MOVC A, @A + DPTR
MOV 61H, A
INC DPTR
CLR A
MOVC A, @A + DPTR
MOV 62H, A
HERE: SJMP HERE

ORG 400H
DB "SAM"
END

; DPTR=401H (points to next location)
; A=0
; get 'A' from location 401H
; move it to RAM location 61H
; DPTR=402H
; A=0
; get 'M' from location 401H
; move it to RAM location 62H
; stay here

; data is burned into location starting from 400H
; directive for end of file

```

Example 17-7

Assuming that ROM space starting at 250H contains "America", write a program to transfer the bytes into RAM locations starting at 40H.

Solution:

(a) This method uses a counter

```

ORG 0000
MOV DPTR, #MYDATA    ;load ROM pointer
MOV R0, #40H         ;load RAM pointer
MOV R2, #7           ;load counter
BACK: CLR A           ;A = 0
      MOVC A, @A + DPTR ;move data from code space
      MOV @R0, A        ;save it in RAM
      INC DPTR          ;increment ROM pointer
      INC R0            ;increment RAM pointer
      DJNZ R2, BACK     ;loop until counter=0
HERE: SJMP HERE

```

-----On-chip code space used for storing data

```

ORG 250H
MYDATA DB "AMERICA"
END

```

(b) This method uses null char for end of string

```

ORG 0000
MOV DPTR, #MYDATA    ;load ROM pointer
MOV R0, #40H         ;load RAM pointer
BACK: CLR A           ;A=0
      MOVC A, @A + DPTR ;move data from code space
      JZ HERE          ;exit if null character
      MOV @R0, A        ;save it in RAM
      INC DPTR          ;increment ROM pointer

```



```

                INC    R0            ;increment RAM pointer
                SJMP   BACK         ;loop
HERE:           SJMP   HERE

; ----- On-chip code space used for storing data
                ORG    250H
MYDATA: DB      "AMERICA", 0        ;notice null char for
                                     ;end of string

                END

```

Notice the null character, 0, indicating the end of the string, and how we use the JZ instruction to detect that.

17.2.8: Look-up table and the MOVC instruction

The look-up table is a widely used concept in microprocessor programming. It allows access to elements of a frequently used table with minimum operations. As an example, assume that for a certain application we need x^2 values in the range of 0 to 9. We can use a look-up table instead of calculating it. This is shown in Example 17-8.

In addition to being used to access program ROM, DPTR can be used to access memory externally connected to the 8051. This is discussed in Chapter 22.

Another register used in indexed addressing mode is the program counter. This is discussed in Appendix A.

In many of the examples above, the MOV instruction was used for the sake of clarity, even though one can use any instruction as long as that instruction supports the addressing mode. For example, the instruction "ADD A, @R0" would add the contents of the memory location pointed to by R0 to the contents of register A. We will see more examples of using addressing modes with various instructions in the next few chapters.

17.2.9: Indexed addressing mode and MOVX instruction

As we have stated earlier, the 8051 has 64K bytes of code space under the direct control of the Program Counter register. We just showed how to use the MOVC instruction to access a portion of this 64K-byte code space as data memory space. In many applications the size of program code does not leave any room to share the 64K-byte code space with data.

Example 17-8

Write a program to get the x value from P1 and send x^2 to P2, continuously.

Solution:

```

                ORG    0
                MOV    DPTR, #300H    ;load look-up table address
                MOV    A, #0FFH       ;A=FF
BACK:           MOV    P1, A          ;configure P1 as input port
                MOV    A, P1          ;get X
                MOVC   A, @A+DPTR      ;get X squared from table
                MOV    P2, A          ;issue it to P2
                SJMP   BACK           ;keep doing it
                ORG    300H
XSQR_TABLE:    DB      0, 1, 4, 9, 16, 25, 36, 49, 64, 81
                END

```

Notice that the first instruction could be replaced with "MOV DPTR, #XSQR_TABLE"

Example 17-9

The common cathode 'seven-segment' codes for the numerals from 0 to 9 are given in the below table. Store this look up table in ROM locations 400H onwards. Write a program to retrieve the contents of these locations one by one, and output it to port P1 which is connected to a seven-segment display which displays the numerals repeatedly with a delay of 100 ms between displays.

Decimal digit	Seven-segment code
0	3F
1	06
2	5B
3	4F
4	66
5	6D
6	7D
7	07
8	7F
9	6F

Solution:

```

START:  ORG 0000H                                ;R1=no of digits, i.e., counter
        MOV R1,#10                               ;load ROM pointer
        MOV DPTR,#400H                           ;A=0
BACK:   CLR A                                     ;move data from code space to A
        MOVC A,@A+DPTR                           ;output data to port 1 for displaying it
        MOV P1,A                                 ;delay of 100 ms
        ACALL DELAY                              ;increment ROM pointer
        INC DPTR                                 ;loop until counter=0
        DJNZ R1, BACK                            ;repeat continuously
        SJMP START

        ORG 0400H                                ;store data at locations starting from 400H
        DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH
        END                                       ;directive for end of file

```

For this reason the 8051 has another 64K bytes of memory space set aside exclusively for data storage. This data memory space is referred to as *external memory* and it is accessed only by the MOVX instruction. In other words, the 8051 has a total of 128K bytes of memory space since 64K bytes of code added to 64K bytes of data space gives us 128K bytes. One major difference between the code space and data space is that, unlike code space, the data space cannot be shared between code and data. This is such an important topic that we have dedicated an entire chapter to it: Chapter 22.

17.2.10: Accessing RAM locations 30 - 7FH as scratch pad

As we have seen so far, in accessing registers R0 - R7 of various banks, it is much easier to refer to them by their R0 - R7 names than by their RAM locations. The only problem is that we have only 4 banks and very often the task of bank switching and keeping track of register bank usage is tedious and prone to errors. For this reason in many applications we use RAM locations 30 - 7FH as scratch pad and leave addresses 8 - 1FH for stack usage. That means that we use R0 - R7 of bank 0, and if we need more registers we simply use RAM locations 30 - 7FH. Look at Example 17-10.

Example 17-10

Write a delay subroutine using RAM location 45H as the counter.

Solution:

```
DELAY:  MOV 45H, #0FFH
REPEAT: DJNZ 45H, REPEAT
        RET
```

;load counter value in RAM location 45H
;decrement count until count=0

17.2.11: Review questions

1. The instruction "MOV A, 40H" uses _____ addressing mode. Why?
2. What address is assigned to register R2 of bank 0?
3. What address is assigned to register R2 of bank 2?
4. What address is assigned to register A?
5. Which registers are allowed to be used for register indirect addressing mode if the data is in on-chip RAM?

SECTION 17.3: BIT ADDRESSES FOR I/O AND RAM

Many microprocessors such as the 386 or Pentium allow programs to access registers and I/O ports in byte size only. In other words, if you need to check a single bit of an I/O port, you must read the entire byte first and then manipulate the whole byte with some logic instructions to get hold of the desired single bit. This is not the case with the 8051. Indeed, one of the most important features of the 8051 is the ability to access the registers, RAM, and I/O ports in bits instead of bytes. This is a very unique and powerful feature for a microprocessor made in the early 1980s. In this section we show address assignment of bits of I/O, register, and memory, in addition to ways of programming them.

17.3.1: Bit-addressable RAM

Of the 128-byte internal RAM of the 8051, only 16 bytes are bit-addressable. The rest must be accessed in byte format. The bit-addressable RAM locations are 20H to 2FH. These 16 bytes provide 128 bits of RAM bit-addressability since $16 \times 8 = 128$. They are addressed as 0 to 127 (in decimal) or 00 to 7FH. Therefore, the bit addresses 0 to 7 are for the first byte of internal RAM location 20H, and 8 to 0FH are the bit addresses of the second byte of RAM location 21H, and so on. The last byte of 2FH has bit addresses of 78H to 7FH. See Fig 17-1 and Example 17-11. Note that internal RAM locations 20 - 2FH are both byte-addressable and bit-addressable.

In order to avoid confusion regarding the addresses 00 - 7FH, the following two points must be noted.

1. The 128 bytes of RAM have the byte addresses of 00 - 7FH and can be accessed in byte size using various addressing modes such as direct and register-indirect, as we have seen in this chapter and previous chapters. These 128 bytes are accessed using byte-type instructions.

Byte address		General-purpose RAM							
7F									
30									
Bit-addressable locations	2F	7F	7E	7D	7C	7B	7A	79	78
	2E	77	76	75	74	73	72	71	70
	2D	6F	6E	6D	6C	6B	6A	69	68
	2C	67	66	65	64	63	62	61	60
	2B	5F	5E	5D	5C	5B	5A	59	58
	2A	57	56	55	54	53	52	51	50
	29	4F	4E	4D	4C	4B	4A	49	48
	28	47	46	45	44	43	42	41	40
	27	3F	3E	3D	3C	3B	3A	39	38
	26	37	36	35	34	33	32	31	30
	25	2F	2E	2D	2C	2B	2A	29	28
	24	27	26	25	24	23	22	21	20
	23	1F	1E	1D	1C	1B	1A	19	18
	22	17	16	15	14	13	12	11	10
	21	0F	0E	0D	0C	0B	0A	09	08
	20	07	06	05	04	03	02	01	00
1F		Bank 3							
18									
17		Bank 2							
10									
0F		Bank 1							
08									
07		Default register bank for R0 - R7							
00									

Figure 17-1. 16 Bytes of internal RAM. Note: They are both bit- and byte-accessible.

Example 17-11

Find out to which byte each of the following bits belongs. Give the address of the RAM byte in hex.

- (a) SETB 42H ;set bit 42H to 1
- (b) CLR 67H ;clear bit 67
- (c) CLR 0FH ;clear bit 0FH

- (d) SETB 28H ;set bit 28H to 1
- (e) CLR 12 ;clear bit 12 (decimal)
- (f) SETB 05

Solution:

- (a) RAM bit address of 42H belongs to D2 of RAM location 28H.
- (b) RAM bit address of 67H belongs to D7 of RAM location 2CH.
- (c) RAM bit address of 0FH belongs to D7 of RAM location 21H.
- (d) RAM bit address of 28H belongs to D0 of RAM location 25H.
- (e) RAM bit address of 12 belongs to D4 of RAM location 21H.
- (f) RAM bit address of 05 belongs to D5 of RAM location 20H.

Table 17-2: Single-Bit Instructions

Instruction	Function
SETB bit	Set the bit (bit = 1)
CLR bit	Clear the bit (bit = 0)
CPL bit	Complement the bit (bit = NOT bit)
JB bit,target	Jump to target if bit = 1 (jump if bit)
JNB bit,target	Jump to target if bit = 0 (jump if no bit)
JBC bit,target	Jump to target if bit = 1, clear bit (jump if bit, then clear)

The 16 bytes of RAM locations 20 - 2FH also have bit addresses of 00 - 7FH since $16 \times 8 = 128$ (00 - 7FH). In order to access these 128 bits of RAM locations and other bit-addressable space of 8051 individually, we can use only the single-bit instructions such as SETB. Table 17-2 provides a list of single-bit instructions. Notice that the single-bit instructions use only one addressing mode and that is direct addressing mode. In the first two sections of this chapter we showed various addressing modes of byte-addressable space of the 8051, among them indirect addressing mode. It must be noted that there is no indirect addressing mode for single-bit instructions.

17.3.2: I/O port bit addresses

As we discussed in Chapter 16, the 8051 has four 8-bit I/O ports: P0, P1, P2, and P3. We can access either the entire 8 bits or any single bit without altering the rest. When accessing a port in a single-bit manner, we use the syntax "SETB P_X.Y" where X is the port number 0, 1, 2, or 3, and Y is the desired bit number from 0 to 7 for data bits D0 to D7. See Fig 17-2. For example, "SETB P1.5" sets high bit 5 of port 1. Remember that D0 is the LSB and D7 is the MSB.

As we mentioned earlier in this chapter, every SFR register is assigned a byte address and ports P0 - P3 are part of the SFR. For example, P0 is assigned byte address 80H, and P1 has address of 90H as shown in Fig 17-2. While all of the SFR registers are byte-addressable some of them are also bit-addressable. The P0 - P3 are among this category of SFR registers. From Fig 17-2 we see that the bit addresses for P0 are 80H to 87H, and for P1 are 90H to 97H, and so on.

Notice that when code such as "SETB P1.0" is assembled, it becomes "SETB 90H" since P1.0 has the RAM address of 90H. Also notice from Figs 17-1 and 17-2 that bit addresses 00 - 7FH belong to RAM byte addresses 20 - 2FH, and bit addresses 80 - F7H belong to SFR of P0, TCON, P1, SCON, P2, etc. The bit addresses for P0 - P3 are shown in Table 17-3, and discussed next.

17.3.3: Bit memory map

From Fig 17-1 and 17-2 and Table 17-3 once again notice the following facts.

1. The bit addresses 00 - 7FH are assigned to RAM locations of 20 - 2FH.
2. The bit addresses 80 - 87H are assigned to the P0 port.
3. The bit addresses 88 - 8FH are assigned to the TCON register.
4. The bit addresses 90 - 97H are assigned to the P1 port.
5. The bit addresses 98 - 9FH are assigned to the SCON register.
6. The bit addresses A0 - A7H are assigned to the P2 port.
7. The bit addresses A8 - AFH are assigned to the IE register.
8. The bit addresses B0 - B7H are assigned to the P3 port.
9. The bit addresses B8 - BFH are assigned to IP.
10. The bit addresses C0 - CFH are not assigned.
11. The bit addresses D0 - D7H are assigned to the PSW register.
12. The bit addresses D8 - DFH are not assigned.
13. The bit addresses E0 - E7H are assigned to the Accumulator register.
14. The bit addresses E8 - EFH are not assigned.
15. The bit addresses F0 - F7H are assigned to the B register.

Byte address	Bit address _s	
FF		
F0	F7 F6 F5 F4 F3 F2 F1 F0	B
E0	E7 E6 E5 E4 E3 E2 E1 E0	ACC
D0	D7 D6 D5 D4 D3 D2 D1 D0	PSW
B8	-- -- -- BC BB BA B9 B8	IE
B0	B7 B6 B5 B4 B3 B2 B1 B0	P3
A8	AF -- -- AC AB AA A9 A8	IP
A0	A7 A6 A5 A4 A3 A2 A1 A0	P2
99	not bit-addressable	
98	9F 9E 9D 9C 9B 9A 99 98	SCON
90	97 96 95 94 93 92 91 90	P1
8D	not bit-addressable	
8C	not bit-addressable	
8B	not bit-addressable	
8A	not bit-addressable	
89	not bit-addressable	
88	8F 8E 8D 8C 8B 8A 89 88	TCON
87	not bit-addressable	PCON
83	not bit-addressable	DPH
82	not bit-addressable	DPL
81	not bit-addressable	SP
80	87 86 85 84 83 82 81 80	P0
Special Function Registers		

Figure 17-2. SFR RAM address (byte and bit)

17.3.4: Registers bit-addressability

While all I/O ports are bit-addressable, that is not the case with registers, as seen from Fig 17-1. Only registers A, B, PSW, IP, IE, ACC, SCON, and TCON are bit-addressable. Of the bit-addressable registers, we will concentrate on the familiar registers A, B, and PSW. The rest will be discussed in future chapters.

Now let's see how we can use bit-addressability of registers such as A and PSW. As we discussed in Chapter 14, in the PSW register two bits are set aside for the selection of the register banks. See Fig 17-3. Upon RESET, bank 0 is selected. We can select any other banks using the bit-addressability of the PSW as was shown in Chapter 14. The bit addressability of PSW also eliminates the need for instructions such as JOV (Jump if OV=1). See Example 17-14.

Examine the next few examples of bit-addressability to gain a better understanding of this important feature of the 8051.

Table 17-3: Bit Addresses for All Ports

P0	Addr	P1	Addr	P2	Addr	P3	Addr	Port's Bit
P0.0	80	P1.0	90	P2.0	A0	P3.0	B0	D0
P0.1	81	P1.1	91	P2.1	A1	P3.1	B1	D1
P0.2	82	P1.2	92	P2.2	A2	P3.2	B2	D2
P0.3	83	P1.3	93	P2.3	A3	P3.3	B3	D3
P0.4	84	P1.4	94	P2.4	A4	P3.4	B4	D4
P0.5	85	P1.5	95	P2.5	A5	P3.5	B5	D5
P0.6	86	P1.6	96	P2.6	A6	P3.6	B6	D6
P0.7	87	P1.7	97	P2.7	A7	P3.7	B7	D7

CY	AC	F0	RS1	RS0	OV	--	P
----	----	----	-----	-----	----	----	---

RS1	RS0	Register Bank	Address
0	0	0	00H - 07H
0	1	1	08H - 0FH
1	0	2	10H - 17H
1	1	3	18H - 1FH

Figure 17-3. Bits of the PSW register

Example 17-12

For each of the following instructions, state to which port the bit belongs. Use Table 17-3.

- (a) SETB 86H (b) CLR 87H (c) SETB 92H (d) SETB 0A7H

Solution:

- (a) SETB 86H is for SETB P0.6.
 (b) CLR 87H is for CLR P0.7.
 (c) SETB 92H is for SETB P1.2.
 (d) SETB 0A7H is for SETB P2.7.

Example 17-13

Write a program to save the Accumulator in R7 of bank 2.

Solution:

```
CLR    PSW.3
SETB   PSW.4
MOV    R7,A
```

Example 17-14

While there are instructions such as JNC and JC to check the carry flag bit (CY), there are no such instructions for the overflow flag bit (OV). How would you write code to check OV?

Solution:

The OV flag is PSW.2 of the PSW register. PSW is a bit-addressable register; therefore, we can use the following instruction to check the OV flag.

```
JB PSW.2,TARGET ;jump if OV=1
```


Example 17-15

Write a program to see if the RAM location 37H contains an even value. If so, send it to P2. If not, make it even and then send it to P2.

Solution:

```
MOV A, 37H      ;load RAM location 37H into accumulator
JNB ACC.0, YES   ;is D0 of reg A 0? if so jump
INC A           ;it is odd, make it even
YES: MOV P2, A   ;send it to P2
```

Example 17-16

The result of a signed arithmetic operation is stored in RAM location 27H. Verify if the stored result is positive or negative. If it is negative send a high value to P1.7, otherwise send a low value.

Solution:

```
MOV A, 27H      ;move the content of RAM location 27H to A
JNB ACC.7, POS   ;check the value of D7 of ACC. If it is set, the sign
                 ;bit shows it to be a negative number
                 ;because it is a negative number, send a high to P1.7
SETB P1.7       ;exit the program
SJMP NEXT
POS: CLR P1.7    ;the sign bit is not set, implies a positive number,
                 ;send a low value to P1.7
NEXT: NOP        ;do nothing on exit
END             ;end of file
```

Example 17-17

The status of bits P1.2 and P1.3 of I/O port P1 must be saved before they are changed. Write a program to save the status of P1.2 in bit location 06 and the status of P1.3 in bit location 07.

Solution:

```
CLR 06          ;clear bit address 06
CLR 07          ;clear bit address 07
JNB P1.2, OVER   ;check bit P1.2, if 0 then jump
SETB 06         ;if P1.2=1, set bit location 06 to 1
OVER: JNB P1.3, NEXT ;check bit P1.3, if 0 then jump
SETB 07         ;if P1.3=1, set bit location 07 to 1
NEXT:...
```

Example 17-18

Write a program to save the status of bit P1.7 on RAM address bit 05.

Solution:

```
MOV C, P1.7      ;get bit from port
MOV 05, C        ;save bit
```

Example 17-19

Move the content of the 7th bit of the A register to pin P0.7, and also save it in RAM location 08H.

Solution:

```
MOV C, ACC.7
MOV P0.7, C
MOV 08, C
```

```
;move the 7th bit of A register to carry
;put it on P0.7
;move to RAM location 08 (bit-addressable area)
```

17.3.5: Using BIT directive

The BIT directive is a widely used directive to assign the bit-addressable I/O and RAM locations. The BIT directive allows a program to assign the I/O or RAM bit at the beginning of the program, making it easier to modify them. Examine the next few examples to see how we use the BIT directive.

Example 17-20

Assume that bit P2.3 is an input and represents the condition of an oven. If it goes high, it means that the oven is hot. Monitor the bit continuously. Whenever it goes high, send a low-to-high pulse to port P1.5 to turn on a buzzer.

Solution:

```
OVEN_HOT BIT P2.3
BUZZER BIT P1.5
```

```
HERE: JNB OVEN_HOT, HERE ;keep monitoring for HOT
      ACALL DELAY ;
      CPL BUZZER ;sound the buzzer
      ACALL DELAY ;
      SJMP HERE
```

This is similar to Example 17-16, except the use of BIT directive allows us to assign the OVEN_HOT and BUZZER bit to any port. This way you do not have to search the program for them.

Example 17-21

An LED is connected to pin P1.7. Write a program to toggle the LED forever.

Solution:

```
LED BIT P.7 ;using BIT directive
HERE: CPL LED ;toggle LED
      LCALL DELAY ;delay
      SJMP HERE ;repeat forever
```

Example 17-22

A switch is connected to pin P1.7 and an LED to pin P2.0. Write a program to get the status of the switch and send it to the LED.

Solution:

```
SW BIT P1.7 ;assign bit
LED BIT P2.0 ;assign bit
HERE: MOV C, SW ;get the bit from the port
      MOV LED, C ;send the bit to the port
      SJMP HERE ;repeat forever
```


Example 17-23

Assume that RAM bit location 12H holds the status of whether there has been a phone call or not. If it is high, it means there has been a new call since it was checked the last time. Write a program to display "New Messages" on an LCD if bit RAM 12H is high. If it is low, the LCD should say "No New Messages".

Solution:

```
PHONBIT BIT 12H
MOV     C, PHONBIT           ;copy bit location 12H to carry
JNC     NO                   ;check to see if is high
MOV     DPTR, #400H          ;yes, load address of message
LCALL   DISPLAY              ;display message (see Chap. 12)
SJMP    EXIT                 ;get out
NO:      MOV     DPTR, #420H   ;load the address of No message
        LCALL   DISPLAY      ;display it
EXIT:
        ;exit
; _____ data to be displayed on LCD
YES_MG:  ORG 400H
        DB  "New Messages", 0
        ORG 420H
NO_MG:   DB  "No New Messages", 0
```

17.3.6: Using EQU directive

We can also use the EQU directive to assign addresses, as shown in the next few examples. Notice that Example 17-24 the ports are defined by their names, while in Example 17-25, they are defined by their addresses.

Example 17-24

The cost prices of 10 items are stored in RAM locations starting from 50H onwards. When these items are sold out as indicated by a 'high' on a switch, the cost price is replaced by the selling price by adding a constant profit of 19 to each value. This changed data is sent out through port 1 with a delay between each data transfer. Write a program for this scenario.

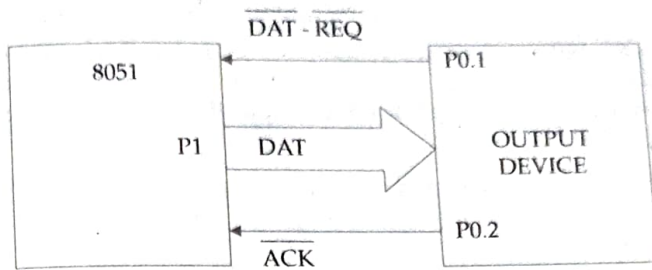
Solution:

```
SW      EQU P0.1
DAT     EQU P1

MOV     C, SW                ;read switch value in to the carry bit
JNC     NEXT                 ;if switch value is not high, exit the loop
MOV     R1, #10              ;counter for number of data items
MOV     R0, #50H             ;R0 is the pointer to the data
BACK:   MOV     A, @R0        ;move the data to the A register
        ADD     A, #19        ;add the profit value
        MOV     @R0, A        ;replace the content of RAM
        MOV     DAT, A        ;send value to P1
        ACALL   DELAY         ;delay
        INC     R0            ;increment pointer value
        DJNZ    R1, BACK      ;check if counter=0
NEXT:   NOP                  ;on exit, do nothing
        END                  ;end of file
```

Example 17-25

When an output device wants data from P1, it sends a 'low' on its 'DAT-REQ' line connected to P0.1. Ten milliseconds after the data is received, the device sends a low on its ACK line connected to P0.2. This indicates the end of one transaction and the 'data request' line is monitored again for a new request. Write a program for this scenario.



Solution:

DAT-REQ EQU 81H
ACK EQU 82H
DAT EQU 90H

```
BACK:    MOV C,DAT-REQ      ;check if there is a request for data
        JC BACK            ;if not low, monitor the DAT-REQ line
        MOV P1,A           ;move data to P1 if DAT-REQ is low
SEND:    ACALL DELAY        ;wait for 10 ms
        JB ACK, HERE       ;check if acknowledgment is received
HERE:    SJMP BACK         ;if yes, await next request for data
```

17.3.7: Review questions

1. True or false. All I/O ports of the 8051 are bit-addressable.
2. True or false. All registers of the 8051 are bit-addressable.
3. True or false. All RAM locations of the 8051 are bit-addressable.
4. Indicate which of the following registers are bit-addressable.
(a) A (b) B (c) R4 (d) PSW (e) R7
5. Of the 128 bytes of RAM in the 8051, how many bytes are bit-addressable? List them.
6. How would you check to see whether bit D0 of R3 is high or low?
7. Find out to which byte each of the following bits belongs. Give the address of the RAM byte in hex.
(a) SETB 20 (b) CLR 32 (c) SETB 12H (d) SETB 95H (e) SETB 0E6H
8. While bit addresses 00 - 7FH belong to _____, bit addresses 80 - F7H belong to _____.
9. True or false. P0, P1, P2, and P3 are part of SFR.
10. True or false. Register ACC is bit-addressable.

SECTION 17.4: EXTRA 128-BYTE ON-CHIP RAM IN 8052

The 8052 microcontroller is an enhanced version of the 8051. In recent years the 8052 has replaced the 8051 due to many of its new features. DS89C420/30 is an example of 8052 architecture. One of the new features of the 8052 is an extra 128 bytes of on-chip RAM space. In other words, the 8051 has only 128 bytes of on-chip RAM, while the 8052 has 256 bytes of it. To understand it, first let's recall the following two facts from earlier discussion in this chapter and Chapter 14.

1. The 8051 has 128 bytes of on-chip RAM with addresses 00 - 7FH. They are used for (a) register banks (addresses 00 - 1FH), (b) bit-addressable RAM space (addresses 20 - 2FH), and (c) the scratch pad (addresses 30 - 7FH).
2. Another 128 bytes of on-chip RAM with addresses 80 - FFH are designated as Special Function Registers (SFRs). Again, the SFRs are accessed by direct addressing mode as we saw earlier in this chapter.

In addition to the above two features, the 8052 has another 128 bytes of on-chip RAM with addresses 80 - FFH. The extra 128 bytes of on-chip RAM is often called upper memory to distinguish it from the lower 128 bytes of 00 - 7FH. The only problem is, the address space 80 - FFH is the same address space assigned to the SFRs. In other words, they are physically two separate memories, but they have the same addresses. This parallel address space in the 8052 forces us to use two different addressing modes to access them as described next.

1. To access the SFRs, we use direct addressing mode. The instruction "MOV 90H, #55H" is an example of accessing the SFR with direct addressing mode. Since 90H is the address of P1, this is same as "MOV P1, #55H".
2. To access the upper 128 bytes, we use the indirect addressing mode, which uses R0 and R1 registers as pointers. Therefore, instructions "MOV @R0, A" and "MOV @R1, A" are employed to access the upper memory as long as registers R0 and R1 have values of 80H or higher. For example, the following codes will put 55H into address 90H of the upper 128 bytes of RAM.

```
MOV R0, #90H      ;load the upper memory address
MOV @R0, #55H     ;put 55H into an address pointed to
                  ;by R0 reg.
```

Fig 17-4 shows the parallel space shared between the SFR and the upper 128 bytes of RAM in the 8052. Example 17-1 shows how to access the upper 128 bytes of on-chip RAM in the 8052 microcontroller.

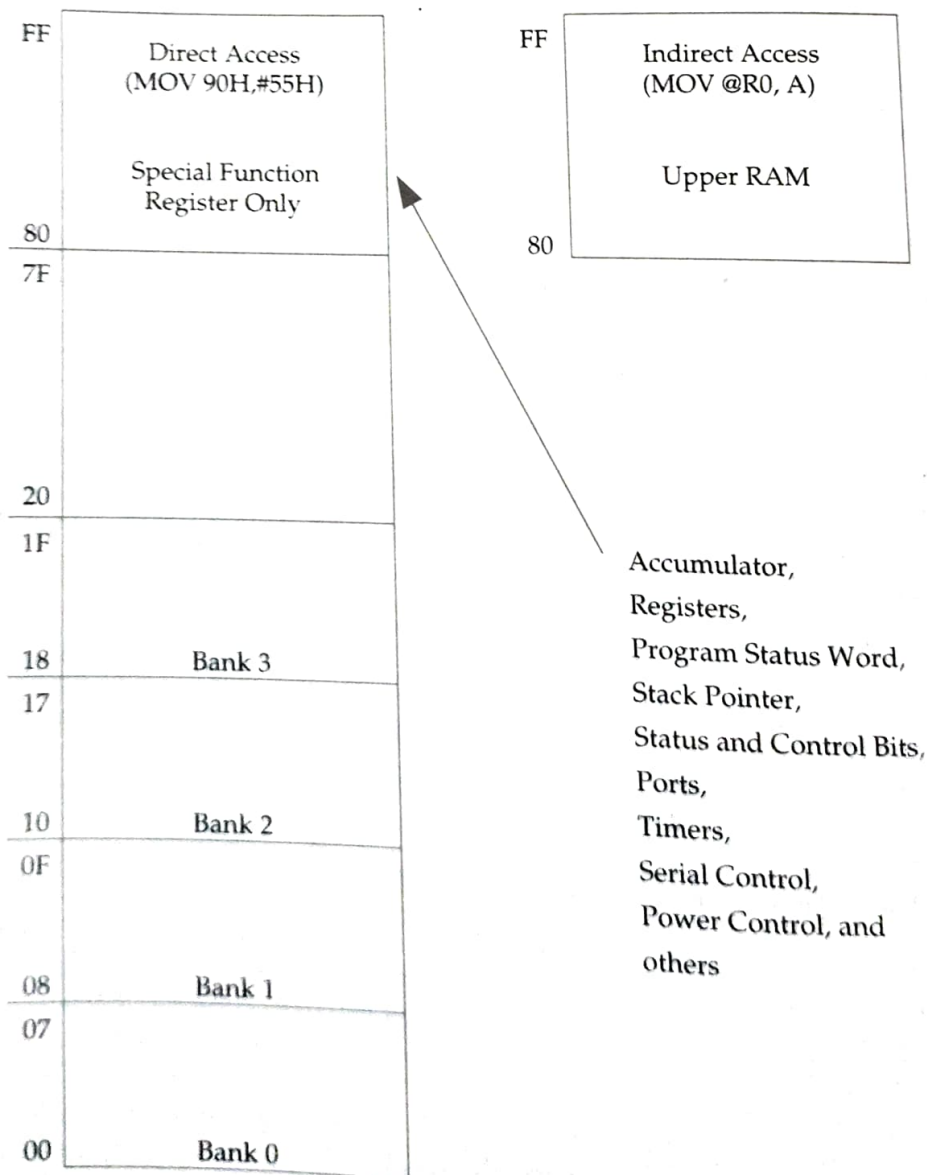


Figure 17-4. 8052 on-chip RAM address space

Example 17-26

Write a program for the 8052 to put 55H into the upper RAM locations of 90 - 99H.

Solution:

```
ORG 0
MOV A, #55H
MOV R2, #10           ;access the upper 128 bytes of on-chip RAM
MOV R0, #90H          ;use indirect addressing mode
MOV @R0, A
BACK: INC R0           ;repeat for all locations
      DJNZ R2, BACK    ;stay here
      SJMP $
      END
```

Run the above program on your simulator and examine the upper memory to see the result. (See Figs 17-5 and 17-6 for screen shots.)

Example 17-27

Assume that the on-chip ROM has a message. Write a program to copy it from code space into the upper memory space starting at address 80H. Also, as you place a byte in upper RAM, give a copy to P0.

Solution:

```
B1:  ORG 0
      MOV DPTR, #MYDATA
      MOV R1, #80H           ;access the upper 128 bytes of on-chip RAM
      CLR A
      MOVC A, @A+DPTR        ;copy from code ROM space
      MOV @R1, A             ;store in upper RAM space
      MOV P0, A              ;give a copy to P0
      JZ EXIT                ;exit if last byte
      INC DPTR               ;increment DPTR
      INC R1                 ;increment R1
      SJMP B1                ;repeat until the last byte
EXIT: SJMP $                 ;stay here when finished
```

```
-----
      ORG 300H
MYDATA: DB "The Promise of World Peace", 0
      END
```

Run the above program on your simulator and examine the upper memory to see the result.

17.4.1: Simulators and data RAM space

All the major 8051/52 simulators have ways of showing the data RAM contents. Figs 17-5 and 17-6 show some of them.

Data (uperspace)									
80:	00	00	00	00	00	00	00	00	
88:	00	00	00	00	00	00	00	00	UUUUUUUU
90:	55	55	55	55	55	55	55	55	UU.....
98:	55	55	00	00	00	00	00	00	
A0:	00	00	00	00	00	00	00	00	
A8:	00	00	00	00	00	00	00	00	
B0:	00	00	00	00	00	00	00	00	
B8:	00	00	00	00	00	00	00	00	
C0:	00	00	00	00	00	00	00	00	
C8:	00	00	00	00	00	00	00	00	
D0:	00	00	00	00	00	00	00	00	
D8:	00	00	00	00	00	00	00	00	
E0:	00	00	00	00	00	00	00	00	
E8:	00	00	00	00	00	00	00	00	
F0:	00	00	00	00	00	00	00	00	
F8:	00	00	00	00	00	00	00	00	

Figure 17-5. Franklin software's ProView upper memory for the 8052

Output Window									
I:0x80	00	00	00	00	00	00	00	00	
I:0x88	00	00	00	00	00	00	00	00	
I:0x90	55	55	55	55	55	55	55	55	UUUUUUUU
I:0x98	55	55	00	00	00	00	00	00	UU.....
I:0xA0	00	00	00	00	00	00	00	00	
I:0xA8	00	00	00	00	00	00	00	00	
I:0xB0	00	00	00	00	00	00	00	00	
I:0xB8	00	00	00	00	00	00	00	00	
I:0xC0	00	00	00	00	00	00	00	00	
I:0xC8	00	00	00	00	00	00	00	00	
I:0xD0	00	00	00	00	00	00	00	00	
I:0xD8	00	00	00	00	00	00	00	00	
I:0xE0	00	00	00	00	00	00	00	00	
I:0xE8	00	00	00	00	00	00	00	00	
I:0xF0	00	00	00	00	00	00	00	00	
I:0xF8	00	00	00	00	00	00	00	00	

>D I:0x80 , 0xFF

<start address> , <end address> <cr>

Build Command Find

Figure 17-6. Keil's µVision upper memory for the 8052

17.4.2: Review questions

1. True or false. The 8052 is an upgraded version of the 8051.
2. True or false. The 8052 has a total of 256 bytes of on-chip RAM in addition to the SFRs.
3. True or false. The extra 128 bytes of RAM in the 8052 is physically the same RAM as the SFR.
4. Give the address for the upper RAM of the 8052.
5. Show how to put value 99H into RAM location F6H of upper RAM in the 8052.

SUMMARY

This chapter described the five addressing modes of the 8051. Immediate addressing mode uses a constant for the source operand. Register addressing mode involves the use of registers to hold data to be manipulated. Direct or register indirect addressing modes can be used to access data stored in either RAM or registers of the 8051. Direct addressing mode is also used to manipulate the stack. Register indirect addressing mode uses a register as a pointer to the data. The advantage of this is that it makes addressing dynamic rather than static. Indexed addressing mode is widely used in accessing data elements of look-up table entries located in the program ROM space of the 8051.

A group of registers called the SFR (special function registers) can be accessed by their names or their addresses. We also discussed the bit-addressable ports, registers, and RAM locations and showed how to use single-bit instructions to access them directly.

PROBLEMS

SECTIONS 17-1 AND 17-2 IMMEDIATE AND REGISTER ADDRESSING MODES/ACCESSING MEMORY USING VARIOUS ADDRESSING MODES

1. What is the difference between the following two instructions in terms of addressing mode and function performed?
MOV A, #46H
MOV A, 46H
2. Write one instruction each using the following addressing modes:
(a) immediate (b) register (c) register indirect (d) direct
3. Indicate the address assigned to each of the following.
(a) R0 of bank 0 (b) ACC (c) R7 of bank 0 (d) R3 of bank 2 (e) B (f) R7 of bank 3
(g) R4 of bank 1 (h) DPL (i) R6 of bank 1 (j) R0 of bank 3 (k) DPH (l) P0
4. Write instructions to push the contents of the following registers onto the stack:
(a) A (b) B (c) R0 of bank 0 (d) R1 of bank 1 (e) R2 of bank 2 (f) R3 of bank 3
5. In accessing the stack, we must use _____ addressing mode.
6. What does the following instruction do? "MOV A, 0F0H"
7. What does the following instruction do? "MOV A, 1FH"
8. Write code to push R0, R1, and R3 of bank 0 onto the stack and pop them back into R5, R6, and R7 of bank 3.
9. What is the error in the following instruction? MOV A, @R2
10. Write a program to copy FFH into RAM locations 50H to 6FH.
11. Write a program to copy 10 bytes of data starting at ROM address 400H to RAM locations starting at 30H.
12. Write a program to find y where $y = x^2 + 2x + 5$, and x is between 0 and 9.
13. Write a program to add the following data and store the result in RAM location 30H.

```
ORG 200H
MYDATA: DB 06, 09, 02, 05, 07
```

SECTION 17-3: BIT ADDRESSES FOR I/O AND RAM

14. "SETB A" is a(n) _____ (valid, invalid) instruction.
15. "CLR A" is a(n) _____ (valid, invalid) instruction.
16. "CPL A" is a(n) _____ (valid, invalid) instruction.
17. True or False? All the I/O ports of 8051 are bit-addressable.
18. Are the registers TCON and TMOD bit-addressable?

19. Write instructions to perform the following actions using bit addresses.
 - (a) set the bit D4 of P1
 - (b) clear the D4 bit of the ACC
 - (c) clear the content of RAM address 67H
 - (d) clear the D3 bit of P0
 - (e) set the D4 bit of PSW.
20. Write a program to generate a square wave with 75% duty cycle on bit P1.5.
21. Write a program to generate a square wave with 80% duty cycle on bit P2.7.
22. Write a program to monitor P1.4. When it goes high, the program will generate a sound (square wave of 500 Hz) on pin P2.7.
23. Write a program to monitor P2.1. When it goes low, the program will send the value 55H to P0.
24. What bit addresses are assigned to P0?
25. What bit addresses are assigned to P1?
26. What bit addresses are assigned to P2?
27. What bit addresses are assigned to P3?
28. What bit addresses are assigned to the PCON register?
29. What bit addresses are assigned to the TCON register?
30. What bit addresses are assigned to register A?
31. What bit addresses are assigned to register B?
32. What bit addresses are assigned to register PSW?
33. What do the following instructions do?
 - (a) SETB 87H
 - (b) CLR 84H
 - (c) SETB 95H
 - (d) CLR 0A3H
34. Write a program to save registers A and B on R3 and R5 of bank 2, respectively.
35. Give another instruction for "CLR C".
36. In Problem 19, assemble each instruction and state if there is any difference between them.
37. What does the following instruction do?
JNB PSW.2, THERE
38. What does the following instruction do?
JB PSW.6, HERE
39. Show how you would check whether the P flag is high.
40. Show how you would check whether the AC flag is high.
41. Give the bit addresses assigned to the flag bit of CY, P, AC, and OV.
42. Of the 128 bytes of RAM locations in the 8051, how many of them are also assigned a bit address as well? which bytes those are.
43. Indicate the bit addresses assigned to RAM locations 20H to 2FH.
44. The byte addresses assigned to the 128 bytes of RAM are _____ to _____.
45. The byte addresses assigned to the SFR are _____ to _____.
46. Indicate the bit addresses assigned to both of the following. Is there a gap between them?
 - (a) RAM locations 20H to 2FH
 - (b) SFR
47. The following are bit addresses. Indicate where each one belongs.

(a) 05H	(b) 47H	(c) 18H	(d) 2DH	(e) 53H	(f) 15H
(g) 67H	(h) 55H	(i) 14H	(j) 37H	(k) 7FH	
48. What is done by the following instructions?
MOV A, P1
JNB ACC.0, YES
49. True or false. The bit addresses of 80H and beyond are assigned to SFR (special function registers)
50. What does the following instruction do?
SETB PSW.4
51. What is the effect of executing the following two instructions?
MOV C, P2.3
MOV 15H, C
52. What is the effect of executing the following two instructions?
MOV C, PSW.0
MOV 13H, C
53. What is the effect of executing the following two instructions?
MOV C, ACC.0
JB HERE
54. Write a program highlighting the use of the EQU directive to denote a port.

- Write a program to show the use of the BIT directive.
- Write a program to set high all the bits of RAM locations 20H to 2FH using the following methods:
 - (a) byte addresses
 - (b) bit addresses
- Write a program to see whether the accumulator is divisible by 8.
- Write a program to find the number of zeros in register R2.

SECTION 17-4: EXTRA 128-BYTE ON-CHIP RAM IN 8052

- What is the total number of bytes of RAM in the 8052 including the SFR registers? Contrast that with the 8051.
- The addressing mode used to access the SFRs of the 8052 is _____, while for addressing the upper 128 bytes of RAM, the _____ addressing mode is used.
- Write an example to show how the upper 128 bytes of 8052 are accessed.
- Give the address range of the lower and the upper 128 bytes of RAM in the 8052.
- In the 8052, the SFR shares the address space with the _____ (lower, upper) 128 bytes of RAM.
- In Question 63, discuss if they are physically the same memory.
- Explain what is the difference between these two instructions.
 - (a) MOV 80H, #99H
 - (b) MOV @R0, #99H if R0=80H
- Write a program to load 10 bytes of data from locations 80H to locations starting at A0 of the upper bank of the 8052.
- Write a program to put 55H into RAM locations C0 - CFH of upper memory.
- Write a program to copy the contents of lower RAM locations 60 - 6FH to upper RAM locations D0 - DFH.

ANSWERS TO REVIEW QUESTIONS

SECTION 17-1: IMMEDIATE AND REGISTER ADDRESSING MODES

- No
- MOV R3, #10000000B
- Source and destination registers' sizes do not match.
- True
- No

SECTION 17-2: ACCESSING MEMORY USING VARIOUS ADDRESSING MODES

- Direct; because there is no "#" sign
- 02
- 12H
- E0H
- R0 and R1

SECTION 17-3: BIT ADDRESSES FOR I/O AND RAM

- True
- False
- False
- A, B, and PSW
- 16 bytes are bit-addressable; they are from byte location 20H to 2FH.
- MOV A, R3
- JNB ACC.0
- For (a), (b), and (c) use Fig 17-1.
 - (a) RAM byte 22H, bit D4
 - (b) RAM byte 24H, bit D0
 - (c) RAM byte 22H, bit D2
- For (d) and (e) use Fig 17-2. (d) SETB P1.5 (e) SETB ACC.6
- RAM bytes 00 - 20H, special function registers.
- True
- True